Optimization — Homework 8— ADMM

Instructor: **Prof. Michael Overton**
Name: **Liyan Gong** Date: **April 3, 2026**

Problems

Implement the ADMM method for the LASSO problem on p. 43 of the ADMM paper by Boyd et al. See p. 32 for the definition of the soft thresholding operator S. My annotated version of the relevant parts of the Boyd paper is here. All of this information is also in my notes. Instead of computing the inverse of $A^TA + \rho I$, solve the relevant system of equations using Cholesky factorization, as explained further below; this is far more efficient for a sparse matrix than computing the inverse. For more details on Cholesky, see BV, p. 669–671 and my notes on Cholesky.

First test the method on a small problem and check whether you get the right answer using CVX. Then fix A to be a randomly generated large but very sparse matrix via `A=sprandn(M,N,density);` (sparse-random-normal distribution) with M=100000, N=10000, density=2/M. Take a look at its sparsity pattern by typing `spy(A),shg` (note that `nz`, at the bottom, means number of nonzeros, which you can also display directly with `nnz(A)`). Set `b=randn(M,1);` and fix ρ (`rho`) to some positive value. Then, before the ADMM iteration starts, compute the *sparse* Cholesky factorization LL^T (BV p. 669) of $A^TA+\rho I$ via `B = A'*A + rho*speye(N); L=chol(B,'lower');` (here, `speye` means sparse identity; do *not* compute the dense identity matrix `eye(N)`). Take a look at L's sparsity with `spy(L),shg`. Then, inside the ADMM iteration you can solve the relevant systems with forward and back substitution (BV, Algorithm C.2, p. 670), using the backslash operator `\`.

For both the small problem and the large problem:

- Experiment with λ : does larger λ result in solutions x which are more sparse, as it should?
- Experiment with ρ : what effect does this have on the method?

Choose the stopping criteria so you get the residuals as small as you can within a reasonable running time (the definition of "reasonable" is entirely up to you!) Submit your MATLAB files, *including comments for full credit*, and relevant log plots showing the behavior of the method, specifically the norms of the primal and dual residuals and the current primal objective value, along with your discussion.

Solutions

Small test problem and CVX verification

We first tested the ADMM method on a small dense problem in order to verify that the implementation is correct before moving to the required large sparse case. We generated a random matrix $A \in \mathbb{R}^{50\times 20}$ with i.i.d. standard normal entries, and constructed a sparse ground-truth vector $x_{\text{true}} \in \mathbb{R}^{20}$ with 5 nonzero entries. The data vector was then defined by

$$b = Ax_{\text{true}} + 0.01\,\xi,$$

where $\xi \sim \mathcal{N}(0, I)$. For this experiment, we used

$$\lambda = 1, \qquad \rho = 1.$$

The ADMM iteration converged in 321 iterations. At termination, the primal objective value was

$$\frac{1}{2}\|Ax - b\|_2^2 + \lambda\|x\|_1 \approx 3.577230.$$

To verify correctness, I also solved the same problem using CVX, which gave

$$p^\star_{\mathrm{CVX}} \approx 3.577321.$$

Thus the relative error in the objective value was

$$\frac{|p_{\mathrm{ADMM}} - p^\star_{\mathrm{CVX}}|}{1 + |p^\star_{\mathrm{CVX}}|} \approx 2.00 \times 10^{-5},$$

and the difference between the two solutions was

$$\|x_{\mathrm{ADMM}} - x_{\mathrm{CVX}}\|_2 \approx 1.82 \times 10^{-4}.$$

The recovered solution was sparse, as expected for the LASSO problem. Using the threshold $|x_i| > 10^{-4}$ to count nonzero entries, both ADMM and CVX produced solutions with 6 nonzero components.

Figure 1 shows the behavior of the method on this small test problem. The top panel shows the primal objective value, which approaches the CVX optimal value. The middle and bottom panels show the primal and dual residual norms, respectively, together with their stopping tolerances. Both residuals decrease steadily, and the stopping criterion is satisfied at iteration 321. Overall, these results confirm that the ADMM implementation is correct on the small test problem.

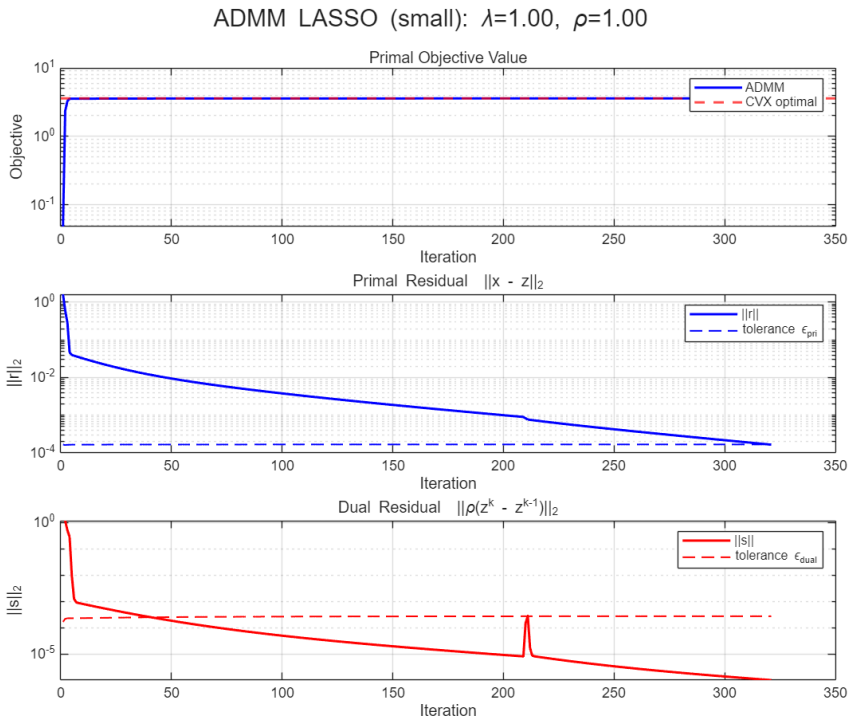


Figure 1: ADMM applied to the small LASSO test problem with $\lambda = 1$ and $\rho = 1$. The top panel shows the primal objective value and the CVX optimum; the middle and bottom panels show the primal and dual residual norms together with their stopping tolerances.

```
%% admm_lasso_small.m
% Solves the LASSO problem via ADMM:
%
%   minimize  (1/2)*||Ax - b||_2^2 + lambda*||x||_1
%
% using the variable splitting x = z, so the scaled ADMM updates are:
%   x-update: x = (A'A + rho*I) \ (A'b + rho*(z - u))   [Cholesky solve]
%   z-update: z = S_{lambda/rho}(x + u)                  [soft thresholding]
%   u-update: u = u + x - z                              [dual variable]
%
% The soft-thresholding operator is:
%   S_kappa(v) = max(v - kappa, 0) - max(-v - kappa, 0)
%
% For the small problem, results are compared against CVX.
% For the large sparse problem, Cholesky factorization is used for efficiency.
%
% Author: [Liyan Gong]
% Date:   April 2026

clear; clc; close all;
rng(42);  % fix random seed for reproducibility

fprintf('=========================================\n');
```

```
=========================================
```

```
fprintf('  ADMM for LASSO -- Small Problem\n');
```

```
  ADMM for LASSO -- Small Problem
```

```
fprintf('=========================================\n\n');
```

```
=========================================
```

```


%% ---- Problem Setup ----
M = 50;   % number of rows (measurements)
N = 20;   % number of columns (variables)

A = randn(M, N);      % random dense measurement matrix
x_true = zeros(N,1);
x_true(1:5) = randn(5,1);   % true signal is 5-sparse
b = A * x_true + 0.01*randn(M,1);  % noisy measurements

lambda = 1.0;   % regularization parameter (try 0.1, 1, 5, 10)
rho    = 1.0;   % ADMM penalty parameter (try 0.1, 1, 5, 10)

fprintf('Problem: M=%d, N=%d, lambda=%.2f, rho=%.2f\n\n', M, N, lambda, rho);
```

```
Problem: M=50, N=20, lambda=1.00, rho=1.00
```

```

```

```
%% ---- Precomputation (done ONCE before iteration) ----
% Form A'A + rho*I and compute its Cholesky factorization L L'
AtA = A' * A;                          % N x N
B   = AtA + rho * eye(N);              % N x N  (dense; ok for small N)
L   = chol(B, 'lower');                % lower-triangular factor: B = L L'
Atb = A' * b;                          % precompute A'b once

%% ---- ADMM Parameters ----
MAX_ITER  = 500;
ABSTOL    = 1e-6;
RELTOL    = 1e-4;

%% ---- Initialization ----
x = zeros(N, 1);
z = zeros(N, 1);
u = zeros(N, 1);   % scaled dual variable  u = (1/rho)*y

%% ---- History storage ----
hist.obj      = zeros(MAX_ITER, 1);
hist.r_norm   = zeros(MAX_ITER, 1);   % primal residual ||x - z||
hist.s_norm   = zeros(MAX_ITER, 1);   % dual   residual ||rho*(z - z_old)||
hist.eps_pri  = zeros(MAX_ITER, 1);
hist.eps_dual = zeros(MAX_ITER, 1);

%% ---- ADMM Iteration ----
fprintf('Iter |  Obj value  |  Primal res |  Dual res\n');
```

```
Iter |  Obj value  |  Primal res |  Dual res
```

```
fprintf('-----|-------------|-------------|----------\n');
```

```
-----|------------|------------|----------
```

```

for k = 1:MAX_ITER

    z_old = z;

    % ---- x-update: solve (A'A + rho*I) x = A'b + rho*(z - u) ----
    % Using the Cholesky factors: L L' x = rhs
    % Step 1: forward substitution  L  w = rhs
    % Step 2: back    substitution  L' x = w
    rhs = Atb + rho * (z - u);
    w   = L  \ rhs;    % forward substitution
    x   = L' \ w;      % back substitution

    % ---- z-update: soft thresholding  S_{lambda/rho}(x + u) ----
    % Soft-threshold operator:
    %   S_kappa(v)_i = sign(v_i) * max(|v_i| - kappa, 0)
    kappa = lambda / rho;
    v     = x + u;
```

```
    z     = sign(v) .* max(abs(v) - kappa, 0);

    % ---- u-update: scaled dual variable ----
    u = u + x - z;

    % ---- Compute residuals (Boyd et al., §3.3) ----
    r = x - z;               % primal residual
    s = -rho * (z - z_old);  % dual residual

    hist.r_norm(k)  = norm(r);
    hist.s_norm(k)  = norm(s);

    % ---- Objective: (1/2)||Ax-b||^2 + lambda||z||_1 ----
    hist.obj(k) = 0.5 * norm(A*x - b)^2 + lambda * norm(z, 1);

    % ---- Stopping criteria ----
    hist.eps_pri(k)  = sqrt(N)*ABSTOL + RELTOL*max(norm(x), norm(z));
    hist.eps_dual(k) = sqrt(N)*ABSTOL + RELTOL*norm(rho*u);

    if mod(k, 50) == 0 || k == 1
        fprintf('%4d | %11.4e | %11.4e | %11.4e\n', ...
            k, hist.obj(k), hist.r_norm(k), hist.s_norm(k));
    end

    if hist.r_norm(k) < hist.eps_pri(k) && hist.s_norm(k) < hist.eps_dual(k)
        fprintf('%4d | %11.4e | %11.4e | %11.4e  --> CONVERGED\n', ...
            k, hist.obj(k), hist.r_norm(k), hist.s_norm(k));
        MAX_ITER = k;
        break;
    end
end
```

```
   1 |  4.7854e-02 |  1.6276e+00 |  0.0000e+00
  50 |  3.5658e+00 |  9.4381e-03 |  1.8441e-04
 100 |  3.5730e+00 |  3.8022e-03 |  4.9794e-05
 150 |  3.5753e+00 |  1.8812e-03 |  1.9676e-05
 200 |  3.5763e+00 |  1.0044e-03 |  9.2207e-06
 250 |  3.5771e+00 |  4.2425e-04 |  3.4111e-06
 300 |  3.5772e+00 |  2.1742e-04 |  1.3976e-06
 321 |  3.5772e+00 |  1.6575e-04 |  1.0187e-06  --> CONVERGED
```

```
%% ---- Trim history ----
hist.obj      = hist.obj(1:MAX_ITER);
hist.r_norm   = hist.r_norm(1:MAX_ITER);
hist.s_norm   = hist.s_norm(1:MAX_ITER);
hist.eps_pri  = hist.eps_pri(1:MAX_ITER);
hist.eps_dual = hist.eps_dual(1:MAX_ITER);

%% ---- CVX comparison ----
fprintf('\nSolving with CVX for ground truth...\n');
```

```
Solving with CVX for ground truth...
```

```
cvx_begin quiet
    variable x_cvx(N)
    minimize( 0.5*sum_square(A*x_cvx - b) + lambda*norm(x_cvx, 1) )
cvx_end
fprintf('CVX optimal value:  %.6f\n', cvx_optval);
```

```
CVX optimal value:  3.577321
```

```
fprintf('ADMM optimal value: %.6f\n', hist.obj(end));
```

```
ADMM optimal value: 3.577230
```

```
fprintf('Relative error in objective: %.2e\n', ...
    abs(hist.obj(end) - cvx_optval) / (1 + abs(cvx_optval)));
```

```
Relative error in objective: 2.00e-05
```

```
fprintf('||x_admm - x_cvx||_2 = %.2e\n', norm(x - x_cvx));
```

```
||x_admm - x_cvx||_2 = 1.82e-04
```

```
%% ---- Sparsity check ----
fprintf('\nNonzeros in ADMM solution (|x|>1e-4): %d / %d\n', ...
    sum(abs(x) > 1e-4), N);
```

```
Nonzeros in ADMM solution (|x|>1e-4): 6 / 20
```

```
fprintf('Nonzeros in CVX  solution (|x|>1e-4): %d / %d\n', ...
    sum(abs(x_cvx) > 1e-4), N);
```

```
Nonzeros in CVX  solution (|x|>1e-4): 6 / 20
```

```
%% ---- Plot convergence ----
K = 1:MAX_ITER;

figure('Name','ADMM LASSO -- Small Problem','Position',[100 100 900 700]);

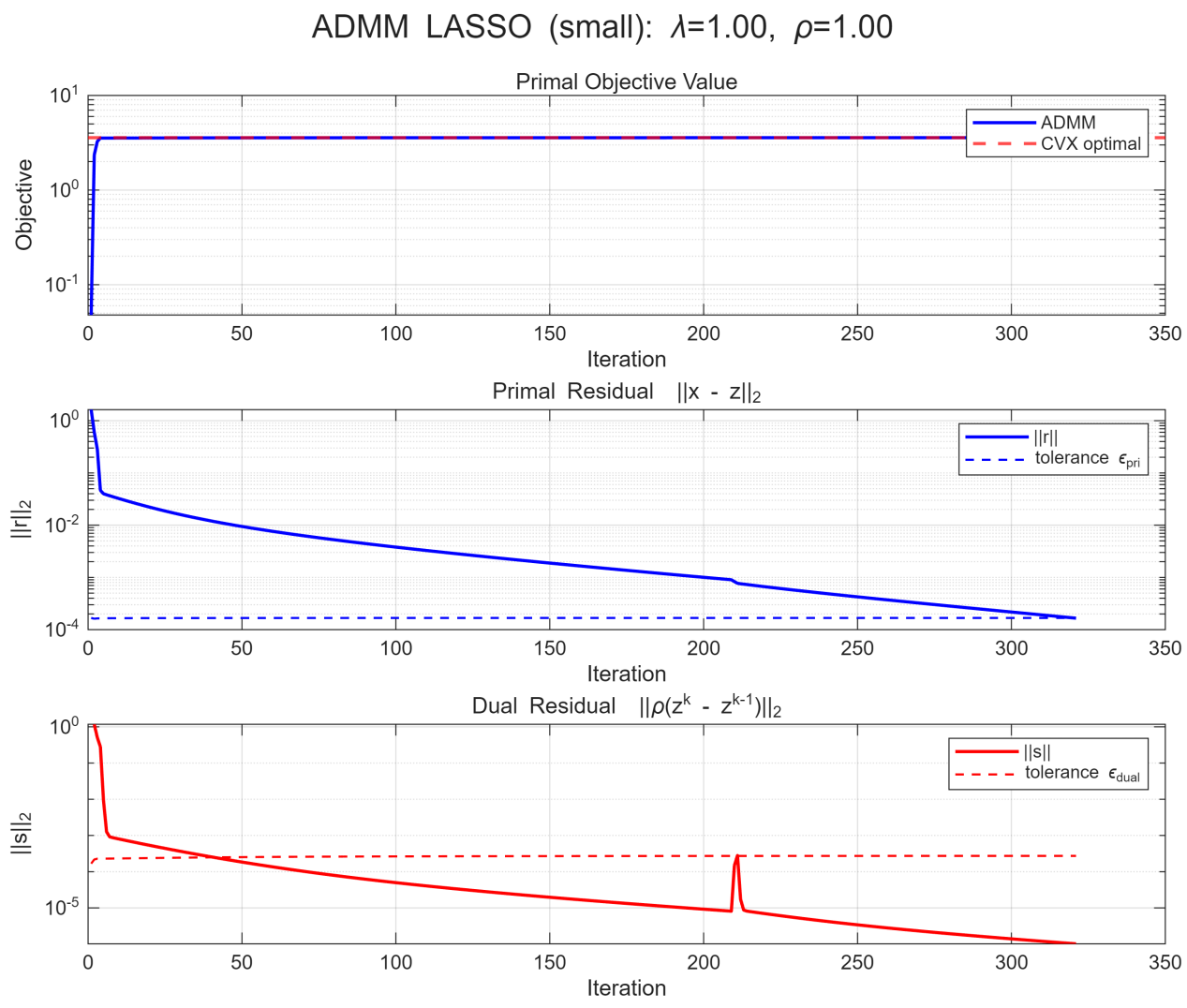
subplot(3,1,1);
semilogy(K, hist.obj, 'b-', 'LineWidth', 1.5); hold on;
yline(cvx_optval, 'r--', 'LineWidth', 1.5);
xlabel('Iteration'); ylabel('Objective');
title('Primal Objective Value');
legend('ADMM','CVX optimal','Location','northeast');
grid on;

subplot(3,1,2);
semilogy(K, hist.r_norm, 'b-', 'LineWidth', 1.5); hold on;
semilogy(K, hist.eps_pri, 'b--', 'LineWidth', 1);
xlabel('Iteration'); ylabel('||r||_2');
title('Primal Residual  ||x - z||_2');
```

```
legend('||r||','tolerance \epsilon_{pri}','Location','northeast');
grid on;

subplot(3,1,3);
semilogy(K, hist.s_norm, 'r-', 'LineWidth', 1.5); hold on;
semilogy(K, hist.eps_dual, 'r--', 'LineWidth', 1);
xlabel('Iteration'); ylabel('||s||_2');
title('Dual Residual  ||\rho(z^k - z^{k-1})||_2');
legend('||s||','tolerance \epsilon_{dual}','Location','northeast');
grid on;

sgtitle(sprintf('ADMM LASSO (small): \\lambda=%.2f, \\rho=%.2f', lambda, rho));
saveas(gcf, 'admm_lasso_small.png');
```




```
fprintf('\nFigure saved: admm_lasso_small.png\n');
```

```
Figure saved: admm_lasso_small.png
```

Large sparse problem

We now apply ADMM to a large-scale sparse LASSO problem, as required in the assignment. The matrix $A \in \mathbb{R}^{100000\times 10000}$ was generated using

$$A = \texttt{sprandn}(M, N, \text{density}), \quad M = 100000,\ N = 10000,\ \text{density} = \frac{2}{M},$$

so that A is extremely sparse, with approximately two nonzero entries per column. In this experiment, we used $\lambda = 1$ and $\rho = 1$.

The sparsity pattern of A is shown in Figure 2, confirming that the matrix is highly sparse. We then formed the matrix

$$B = A^T A + \rho I,$$

and computed its sparse Cholesky factorization $B = LL^T$. The sparsity pattern of L is shown in Figure 3. We observe that L remains sparse, although slightly denser than A, as expected due to fill-in.

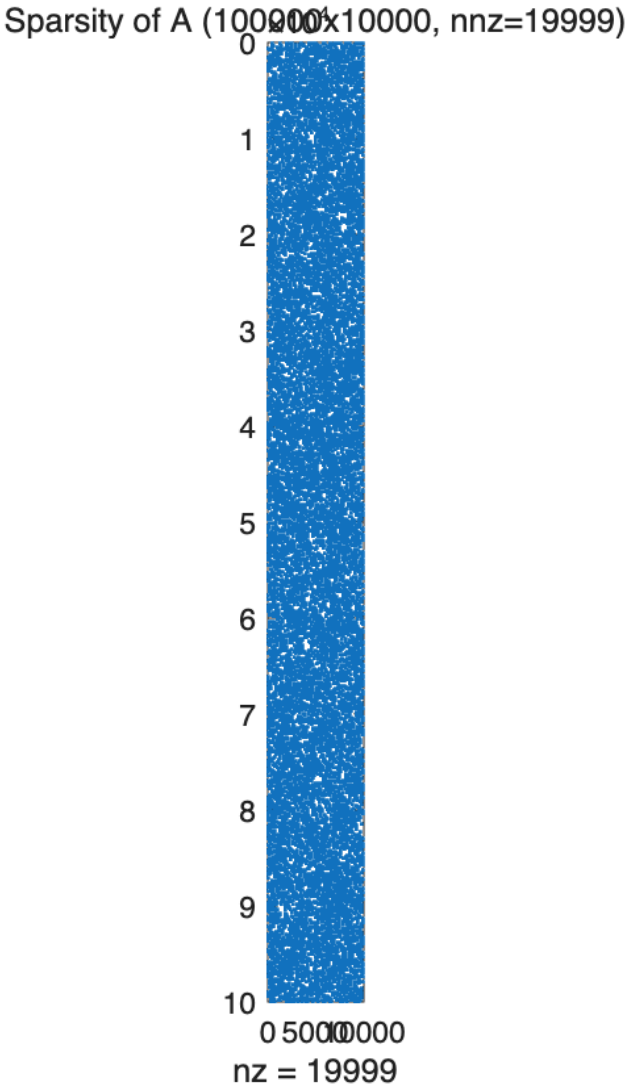


Figure 2: Sparsity pattern of the matrix A.

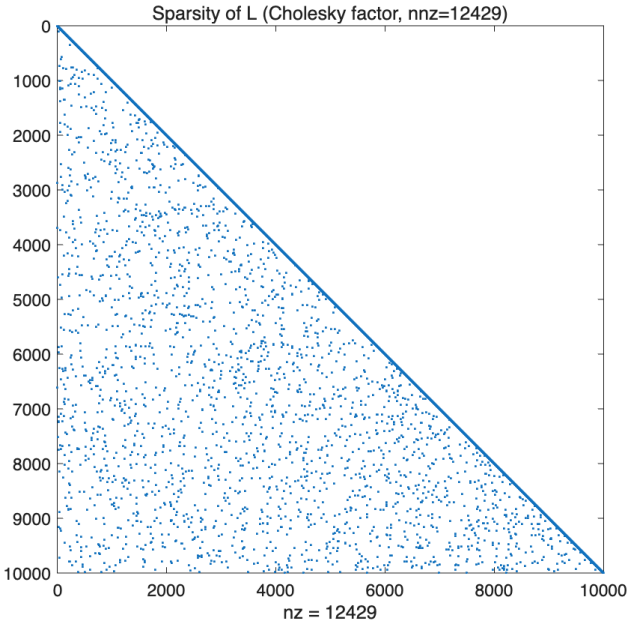


Figure 3: Sparsity pattern of the Cholesky factor L of $A^TA + \rho I$.

The ADMM method converged in 31 iterations, with a total runtime of approximately 0.03 seconds. At termination, the primal objective value was

$$\frac{1}{2}\|Ax - b\|_2^2 + \lambda\|x\|_1 \approx 4.906666 \times 10^4.$$

The primal and dual residuals decreased rapidly and satisfied the stopping criteria, indicating fast convergence of the method.

The recovered solution was moderately sparse. Using the threshold $|x_i| > 10^{-4}$, the solution had

$$3794 \text{ nonzero entries out of } 10000 \ (\approx 37.9\%).$$

This reflects the effect of the ℓ_1 regularization: while the solution is not extremely sparse for $\lambda = 1$, many coefficients are driven to zero.

Figure 4 shows the convergence behavior of the ADMM method for this problem. The objective value stabilizes quickly, and both the primal and dual residuals decrease rapidly, reaching their respective tolerances within a small number of iterations. This demonstrates that ADMM, combined with sparse Cholesky factorization, is highly efficient for large-scale sparse LASSO problems.

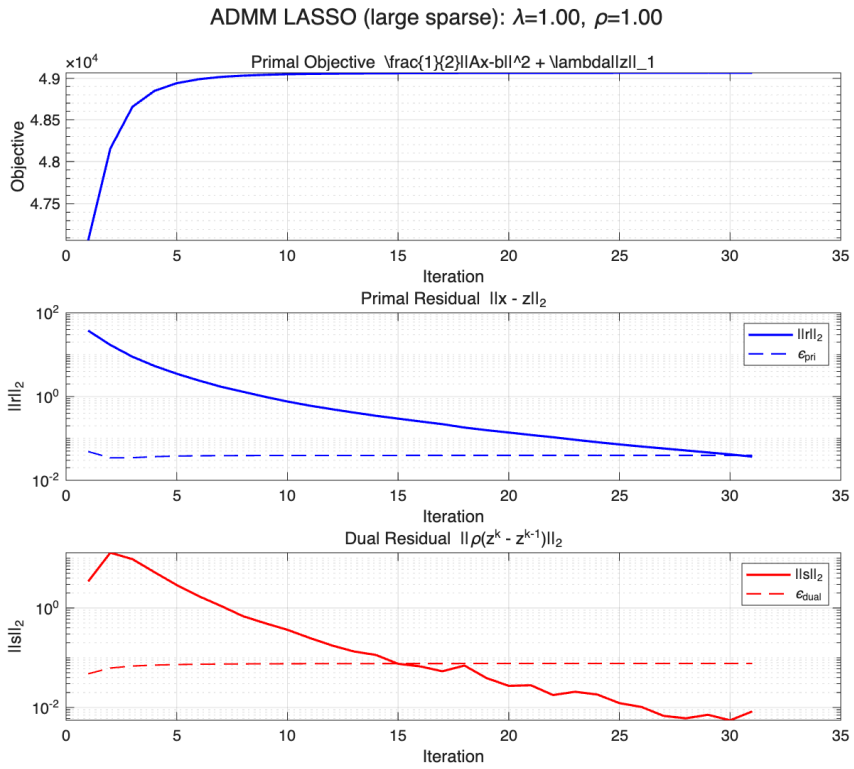


Figure 4: ADMM convergence behavior for the large sparse LASSO problem.

Table of Contents

admm_lasso_large.m 1
---- Problem Setup ---- 1
---- Parameters to experiment with ---- 2
---- Precompute Sparse Cholesky ONCE (outside the iteration) ---- 2
---- ADMM Parameters ---- 4
---- Initialization ---- 4
---- History ---- 4
---- ADMM Iteration ---- 4
---- Trim history ---- 5
---- Report solution sparsity ---- 6
---- Log plots ---- 6

admm_lasso_large.m

Solves the LASSO problem via ADMM on a LARGE, VERY SPARSE matrix A generated by sprandn.

```
minimize  (1/2)*||Ax - b||_2^2 + lambda*||x||_1
```

Author: [Liyan Gong] Date: April 2026

```
clear; clc; close all;
rng(42);

fprintf('========================================\n');
fprintf('  ADMM for LASSO -- Large Sparse Problem\n');
fprintf('========================================\n\n');
```

```
========================================
  ADMM for LASSO -- Large Sparse Problem
========================================
```

---- Problem Setup ----

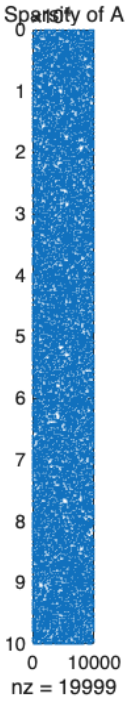
```
M       = 100000;    % rows
N       = 10000;     % columns
density = 2/M;       % very sparse: ~2 nonzeros per column on average

fprintf('Generating sparse matrix A (%d x %d, density=%.1e)...\n', M, N,
density);
A = sprandn(M, N, density);    % sparse random normal
b = randn(M, 1);

fprintf('nnz(A) = %d  (%.4f%% fill)\n\n', nnz(A), 100*nnz(A)/(M*N));

% Visualize sparsity pattern of A
figure('Name','Sparsity of A');
spy(A); title(sprintf('Sparsity of A'));
saveas(gcf, 'spy_A.png');
```

```
Generating sparse matrix A (100000 x 10000, density=2.0e-05)...
nnz(A) = 19999  (0.0020% fill)
```



---- Parameters to experiment with ----

Try multiple (lambda, rho) combinations: - Larger lambda => sparser solution x - rho affects convergence speed but not the optimal value

```
lambda = 1.0;    % regularization  (try 0.1, 1, 5, 10, 50)
rho    = 1.0;    % ADMM penalty    (try 0.1, 1, 5, 10)

fprintf('Parameters: lambda=%.2f, rho=%.2f\n\n', lambda, rho);
```

```
Parameters: lambda=1.00, rho=1.00
```

---- Precompute Sparse Cholesky ONCE (outside the iteration) ----

B = A'A + rho*I is N x N (10000 x 10000), symmetric positive definite. We use speye(N) (SPARSE identity) -- do NOT use eye(N) which is dense! MATLAB's chol with 'lower' gives L such that B = L * L'. This exploits the sparsity of A to keep L sparse as well.

```
fprintf('Computing B = A''A + rho*speye(N) ...\n');
tic;
B = A'*A + rho * speye(N);    % sparse N x N SPD matrix
```

```
t_AtA = toc;
fprintf('  A''A + rho*I computed in %.2f sec, nnz(B)=%d\n', t_AtA, nnz(B));

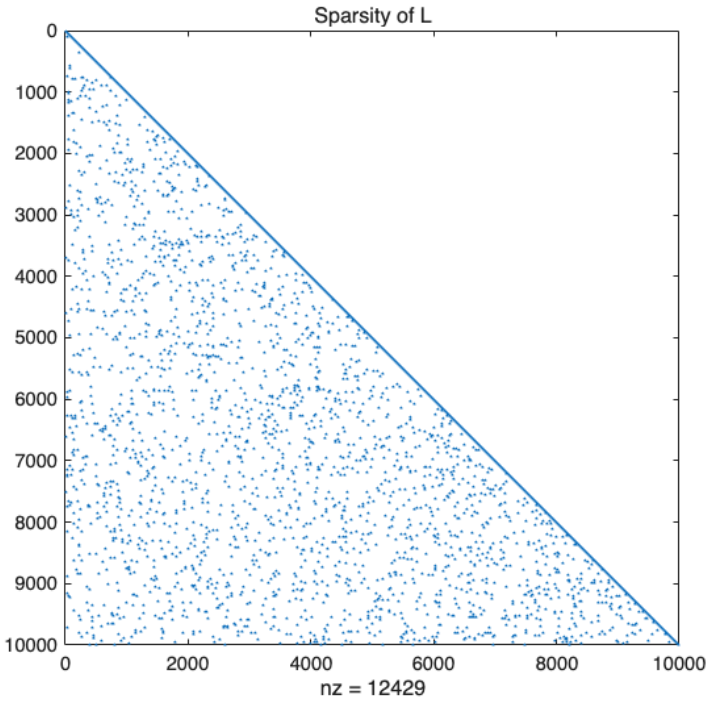
fprintf('Computing sparse Cholesky factorization L (B = LL'') ...\n');
tic;
L = chol(B, 'lower');        % sparse lower-triangular Cholesky factor
t_chol = toc;
fprintf('  Cholesky done in %.2f sec, nnz(L)=%d\n\n', t_chol, nnz(L));

% Visualize sparsity of L
figure('Name','Sparsity of L');
spy(L); title(sprintf('Sparsity of L', nnz(L)));
saveas(gcf, 'spy_L.png');

% Precompute A'b once
fprintf('Precomputing A''b ...\n');
Atb = A' * b;
```

```
Computing B = A'A + rho*speye(N) ...
  A'A + rho*I computed in 0.00 sec, nnz(B)=14114
Computing sparse Cholesky factorization L (B = LL') ...
  Cholesky done in 0.00 sec, nnz(L)=12429

Precomputing A'b ...
```



---- ADMM Parameters ----

```
MAX_ITER = 200;
ABSTOL   = 1e-4;    % looser tolerance acceptable for large problem
RELTOL   = 1e-3;
```

---- Initialization ----

```
x = zeros(N, 1);
z = zeros(N, 1);
u = zeros(N, 1);
```

---- History ----

```
hist.obj    = zeros(MAX_ITER, 1);
hist.r_norm = zeros(MAX_ITER, 1);
hist.s_norm = zeros(MAX_ITER, 1);
hist.eps_pri  = zeros(MAX_ITER, 1);
hist.eps_dual = zeros(MAX_ITER, 1);
hist.time   = zeros(MAX_ITER, 1);
```

---- ADMM Iteration ----

```
fprintf('Starting ADMM iteration...\n');
fprintf('Iter |  Obj value  |  Primal res |  Dual res   | Time(s)\n');
fprintf('-----|-------------|-------------|-------------|--------\n');

t_start = tic;

for k = 1:MAX_ITER

    z_old = z;

    % ---- x-update ----
    % Solve: (A'A + rho*I) x = A'b + rho*(z - u)
    % Using precomputed L: B = L*L', so:
    %   L  w = rhs   (forward substitution, BV p.670 step 1)
    %   L' x = w     (back    substitution, BV p.670 step 2)
    rhs = Atb + rho * (z - u);
    w   = L  \ rhs;    % forward substitution: O(nnz(L))
    x   = L' \ w;      % back    substitution: O(nnz(L))

    % ---- z-update: soft thresholding ----
    % S_kappa(v)_i = sign(v_i) * max(|v_i| - kappa, 0)
    kappa = lambda / rho;
    v     = x + u;
    z     = sign(v) .* max(abs(v) - kappa, 0);

    % ---- u-update: scaled dual variable ----
    u = u + x - z;
```

```
    % ---- Residuals ----
    r = x - z;
    s = -rho * (z - z_old);

    hist.r_norm(k)  = norm(r);
    hist.s_norm(k)  = norm(s);

    % Objective: (1/2)||Ax-b||^2 + lambda||z||_1
    hist.obj(k)  = 0.5 * norm(A*x - b)^2 + lambda * norm(z, 1);
    hist.time(k) = toc(t_start);

    % ---- Stopping criteria ----
    hist.eps_pri(k)  = sqrt(N)*ABSTOL + RELTOL*max(norm(x), norm(z));
    hist.eps_dual(k) = sqrt(N)*ABSTOL + RELTOL*norm(rho*u);

    if mod(k, 10) == 0 || k == 1
        fprintf('%4d | %11.4e | %11.4e | %11.4e | %.2f\n', ...
            k, hist.obj(k), hist.r_norm(k), hist.s_norm(k), hist.time(k));
    end

    if hist.r_norm(k) < hist.eps_pri(k) && hist.s_norm(k) < hist.eps_dual(k)
        fprintf('%4d | %11.4e | %11.4e | %11.4e | %.2f  --> CONVERGED\n', ...
            k, hist.obj(k), hist.r_norm(k), hist.s_norm(k), hist.time(k));
        MAX_ITER = k;
        break;
    end
end

total_time = toc(t_start);
fprintf('\nTotal ADMM time: %.2f sec over %d iterations\n', total_time,
MAX_ITER);
```

```
Starting ADMM iteration...
Iter |  Obj value   |  Primal res |  Dual res   | Time(s)
-----|--------------|-------------|-------------|--------
   1 |  4.7071e+04 |  3.7512e+01 |  3.4230e+00 | 0.00
  10 |  4.9049e+04 |  7.6074e-01 |  3.6461e-01 | 0.01
  20 |  4.9065e+04 |  1.3833e-01 |  2.7181e-02 | 0.02
  30 |  4.9067e+04 |  4.1632e-02 |  5.5220e-03 | 0.02
  31 |  4.9067e+04 |  3.6752e-02 |  8.3192e-03 | 0.02  --> CONVERGED

Total ADMM time: 0.02 sec over 31 iterations
```

---- Trim history ----

```
hist.obj      = hist.obj(1:MAX_ITER);
hist.r_norm   = hist.r_norm(1:MAX_ITER);
hist.s_norm   = hist.s_norm(1:MAX_ITER);
hist.eps_pri  = hist.eps_pri(1:MAX_ITER);
hist.eps_dual = hist.eps_dual(1:MAX_ITER);
hist.time     = hist.time(1:MAX_ITER);
```

---- Report solution sparsity ----

```
nnz_x = sum(abs(x) > 1e-4);
fprintf('\nSparsity of solution x (|x_i|>1e-4): %d / %d nonzeros (%.1f%%)
\n', ...
    nnz_x, N, 100*nnz_x/N);
fprintf('Primal objective at termination: %.6e\n', hist.obj(end));
```

```
Sparsity of solution x (|x_i|>1e-4): 3794 / 10000 nonzeros (37.9%)
Primal objective at termination: 4.906666e+04
```

---- Log plots ----

```
K = 1:MAX_ITER;

figure('Name','ADMM LASSO -- Large Problem','Position',[100 100 900 750]);

subplot(3,1,1);
semilogy(K, hist.obj, 'b-', 'LineWidth', 1.5);
xlabel('Iteration'); ylabel('Objective');
title('Primal Objective  \frac{1}{2}||Ax-b||^2 + \lambda||z||_1');
grid on;

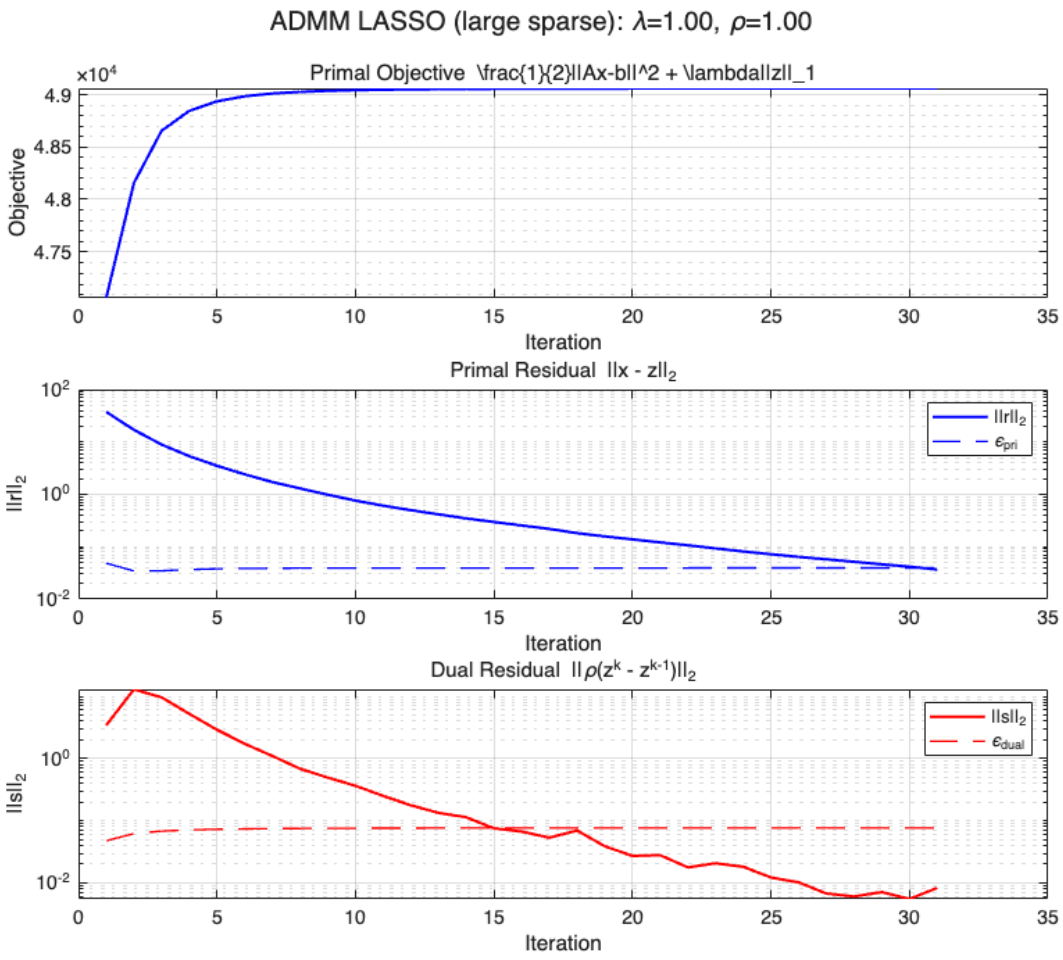
subplot(3,1,2);
semilogy(K, hist.r_norm, 'b-', 'LineWidth', 1.5); hold on;
semilogy(K, hist.eps_pri, 'b--', 'LineWidth', 1);
xlabel('Iteration'); ylabel('||r||_2');
title('Primal Residual  ||x - z||_2');
legend('||r||_2', '\epsilon_{pri}', 'Location', 'northeast');
grid on;

subplot(3,1,3);
semilogy(K, hist.s_norm, 'r-', 'LineWidth', 1.5); hold on;
semilogy(K, hist.eps_dual, 'r--', 'LineWidth', 1);
xlabel('Iteration'); ylabel('||s||_2');
title('Dual Residual  ||\rho(z^k - z^{k-1})||_2');
legend('||s||_2', '\epsilon_{dual}', 'Location', 'northeast');
grid on;

sgtitle(sprintf('ADMM LASSO (large sparse): \\lambda=%.2f, \\rho=%.2f',
lambda, rho));
saveas(gcf, 'admm_lasso_large.png');
fprintf('Figure saved: admm_lasso_large.png\n');
```

```
警告: SceneNode 状□□□。
字符串□量或字符向量必□具□有效的解□器□法:
Primal Objective  \\frac{1}{2}||Ax-b||^2 + \\lambda||z||_1

Figure saved: admm_lasso_large.png
```



Published with MATLAB® R2025b

Effect of λ and ρ

We conducted numerical experiments to study the effect of the parameters λ and ρ on the ADMM method for the LASSO problem.

Effect of λ. We first fixed $\rho = 1$ and varied λ. Table 1 summarizes the results, including the final objective value, the number of nonzero components in the solution (using the threshold $|x_i| > 10^{-4}$), and the number of iterations required for convergence.

λ	Objective value	nnz($\lvert x_i\rvert > 10^{-4}$)	Iterations
0.01	3.8315×10^{-2}	18	197
0.1	3.6511×10^{-1}	5	668
1	3.5773×10^{0}	6	697
5	1.6682×10^{1}	6	848
10	3.0371×10^{1}	6	925
50	6.6739×10^{1}	1	1000

Table 1: Effect of λ on the solution sparsity and convergence behavior (with $\rho = 1$).

As λ increases, the ℓ_1 regularization becomes stronger, which promotes sparsity in the solution. This is clearly observed in the table: for small values of λ, most components of x are nonzero, while for large λ, many components are driven to zero (e.g., only one nonzero entry when $\lambda = 50$).

Figure 5 further illustrates the convergence behavior. While larger λ leads to sparser solutions, it also tends to slow down convergence, as reflected by the increased number of iterations required.

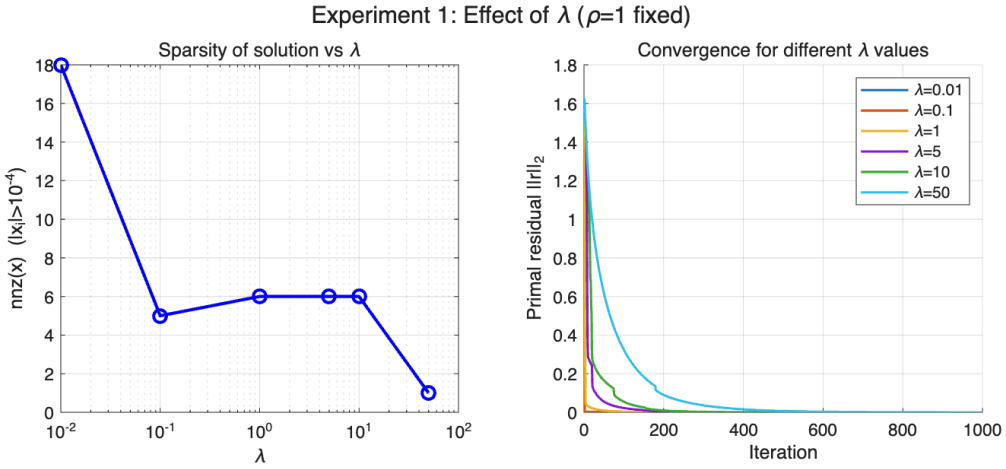


Figure 5: Effect of λ on the convergence of ADMM (with $\rho = 1$).

Effect of ρ. Next, we fixed $\lambda = 1$ and varied ρ. The results are summarized in Table 2.

The parameter ρ does not affect the optimal solution of the LASSO problem, but it has a strong influence on the convergence speed of ADMM. For small values of ρ, convergence is very slow and may not be achieved within the maximum number of iterations. As ρ increases, the convergence becomes significantly faster.

In this experiment, larger values of ρ (such as $\rho = 10$ and $\rho = 50$) lead to much faster convergence, reducing the number of iterations dramatically.

Figure 6 shows the convergence behavior for different values of ρ. We observe that appropriate choices of ρ can greatly improve the efficiency of the ADMM method.

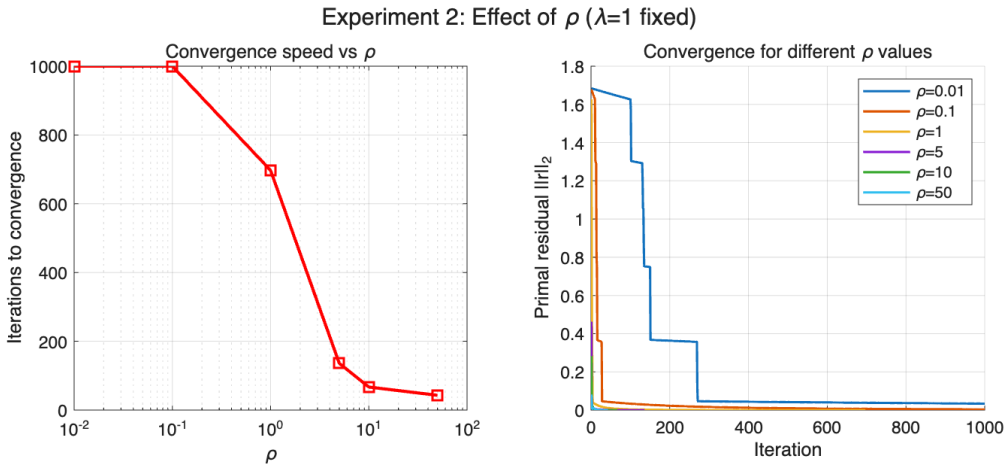


Figure 6: Effect of ρ on the convergence of ADMM (with $\lambda = 1$).

Summary. These experiments demonstrate that:

- The parameter λ controls the sparsity of the solution: larger λ leads to sparser solutions.
- The parameter ρ does not change the optimal solution, but strongly affects the convergence speed.
- Very small values of ρ lead to slow convergence, while moderately large values significantly accelerate the method.

Table of Contents

admm_lasso_experiments.m 1
---- Problem Setup (small) ---- 1
---- Helper: run ADMM and return history ---- 1
========== 2
========== 3
Plot: lambda effect on sparsity 3
========== 4
========== 4
Plot: rho effect on convergence 5

admm_lasso_experiments.m

Experiments studying the effect of lambda and rho on ADMM for LASSO.

Experiment 1: Vary lambda -- does larger lambda give sparser solutions? Experiment 2: Vary rho -- what effect on convergence speed?

Uses the SMALL problem for quick experiments.

Author: [Liyan Gong] Date: April 2026

```
clear; clc; close all;
rng(42);

fprintf('================================================\n');
fprintf('  ADMM LASSO -- Lambda and Rho Experiments\n');
fprintf('================================================\n\n');
```

```
================================================
  ADMM LASSO -- Lambda and Rho Experiments
================================================
```

---- Problem Setup (small) ----

```
M = 50; N = 20;
A = randn(M, N);
x_true = zeros(N,1); x_true(1:5) = randn(5,1);
b = A * x_true + 0.01*randn(M,1);

MAX_ITER = 1000;
ABSTOL   = 1e-8;
RELTOL   = 1e-6;
```

---- Helper: run ADMM and return history ----

```
function [x, hist, iters] = run_admm(A, b, lambda, rho, MAX_ITER, ABSTOL,
RELTOL)
```

```
    [~, N] = size(A);
    Atb = A' * b;
    B = A'*A + rho*eye(N);
    L = chol(B, 'lower');
    x = zeros(N,1); z = zeros(N,1); u = zeros(N,1);
    hist.obj    = zeros(MAX_ITER,1);
    hist.r_norm = zeros(MAX_ITER,1);
    hist.s_norm = zeros(MAX_ITER,1);
    for k = 1:MAX_ITER
        z_old = z;
        rhs = Atb + rho*(z - u);
        x = L' \ (L \ rhs);
        kappa = lambda/rho;
        v = x + u;
        z = sign(v).*max(abs(v)-kappa, 0);
        u = u + x - z;
        hist.r_norm(k) = norm(x - z);
        hist.s_norm(k) = norm(-rho*(z - z_old));
        hist.obj(k)    = 0.5*norm(A*x-b)^2 + lambda*norm(z,1);
        eps_pri  = sqrt(N)*ABSTOL + RELTOL*max(norm(x),norm(z));
        eps_dual = sqrt(N)*ABSTOL + RELTOL*norm(rho*u);
        if hist.r_norm(k) < eps_pri && hist.s_norm(k) < eps_dual
            iters = k; hist.obj = hist.obj(1:k);
            hist.r_norm = hist.r_norm(1:k);
            hist.s_norm = hist.s_norm(1:k);
            return;
        end
    end
    iters = MAX_ITER;
    hist.obj    = hist.obj(1:MAX_ITER);
    hist.r_norm = hist.r_norm(1:MAX_ITER);
    hist.s_norm = hist.s_norm(1:MAX_ITER);
end
```

0.01	*3.8315e-02*	*18*	*197*
0.1	*3.6511e-01*	*5*	*668*
1	*3.5773e+00*	*6*	*697*
5	*1.6682e+01*	*6*	*848*
10	*3.0371e+01*	*6*	*925*
50	*6.6739e+01*	*1*	*1000*

==========

==========

```
EXPERIMENT 1: Vary lambda, fix rho
Hypothesis: larger lambda => sparser x (more zeros)
```

```
================================================================

======================
```

```
fprintf('Experiment 1: Effect of lambda (rho=1 fixed)\n');
fprintf('-------------------------------------------\n');
lambdas = [0.01, 0.1, 1, 5, 10, 50];
rho_fixed = 1.0;

fprintf('%-10s %-12s %-12s %-10s\n','lambda','Obj
value','nnz(x>1e-4)','Iters');
lambda_results = struct();
for i = 1:length(lambdas)
    lam = lambdas(i);
    [x, hist, iters] = run_admm(A, b, lam, rho_fixed, MAX_ITER, ABSTOL,
RELTOL);
    nnz_x = sum(abs(x) > 1e-4);
    fprintf('%-10.3g %-12.4e %-12d %-10d\n', lam, hist.obj(end), nnz_x,
iters);
    lambda_results(i).lambda = lam;
    lambda_results(i).obj    = hist.obj(end);
    lambda_results(i).nnz    = nnz_x;
    lambda_results(i).iters  = iters;
    lambda_results(i).r_norm = hist.r_norm;
    lambda_results(i).s_norm = hist.s_norm;
    lambda_results(i).obj_hist = hist.obj;
end
```

```
Experiment 1: Effect of lambda (rho=1 fixed)
-------------------------------------------
lambda     Obj value    nnz(x>1e-4)  Iters
```

Plot: lambda effect on sparsity

```
figure('Name','Effect of Lambda','Position',[50 50 1000 400]);

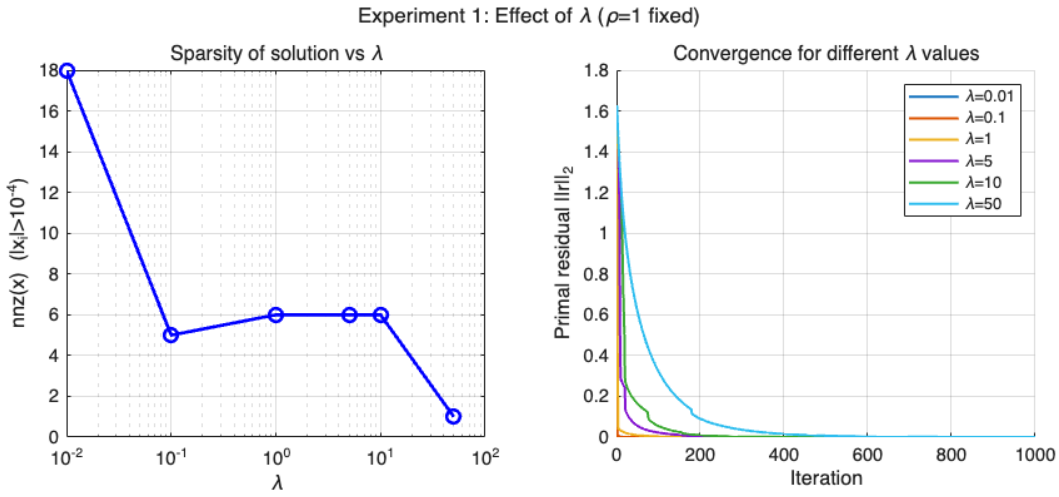
subplot(1,2,1);
nnz_vals = [lambda_results.nnz];
semilogx([lambda_results.lambda], nnz_vals, 'bo-', 'LineWidth', 2,
'MarkerSize', 8);
xlabel('\lambda'); ylabel('nnz(x)  (|x_i|>10^{-4})');
title('Sparsity of solution vs \lambda');
grid on;

subplot(1,2,2);
colors = lines(length(lambdas));
hold on;
for i = 1:length(lambdas)
    semilogy(lambda_results(i).r_norm, 'Color', colors(i,:), 'LineWidth',
1.5);
end
xlabel('Iteration'); ylabel('Primal residual ||r||_2');
```

```
title('Convergence for different \lambda values');
legend(arrayfun(@(l) sprintf('\\lambda=%.2g',l), lambdas,
'UniformOutput',false), ...
    'Location','northeast');
grid on;

sgtitle('Experiment 1: Effect of \lambda (\rho=1 fixed)');
saveas(gcf, 'experiment_lambda.png');
fprintf('Figure saved: experiment_lambda.png\n\n');
```

Figure saved: experiment_lambda.png




```
================================================================================

============================
```

```
EXPERIMENT 2: Vary rho, fix lambda
Effect: rho controls penalty strength and convergence rate
but NOT the optimal value (rho appears nowhere in the objective)
```

```
================================================================================

============================
```

```
fprintf('\nExperiment 2: Effect of rho (lambda=1 fixed)\n');
fprintf('-----------------------------------------------\n');
rhos = [0.01, 0.1, 1, 5, 10, 50];
lambda_fixed = 1.0;

fprintf('%-10s %-12s %-12s %-10s\n','rho','Obj value','nnz(x>1e-4)','Iters');
rho_results = struct();
for i = 1:length(rhos)
    rho = rhos(i);
    [x, hist, iters] = run_admm(A, b, lambda_fixed, rho, MAX_ITER, ABSTOL,
RELTOL);
```

```
    nnz_x = sum(abs(x) > 1e-4);
    fprintf('%-10.3g %-12.4e %-12d %-10d\n', rho, hist.obj(end), nnz_x,
iters);
    rho_results(i).rho    = rho;
    rho_results(i).obj    = hist.obj(end);
    rho_results(i).nnz    = nnz_x;
    rho_results(i).iters  = iters;
    rho_results(i).r_norm = hist.r_norm;
    rho_results(i).s_norm = hist.s_norm;
end


Experiment 2: Effect of rho (lambda=1 fixed)
--------------------------------------------
rho         Obj value    nnz(x>1e-4)  Iters
0.01        3.5476e+00   19           1000
0.1         3.5730e+00   20           1000
1           3.5773e+00   6            697
5           3.5773e+00   6            136
10          3.5773e+00   6            67
50          3.5773e+00   6            43
```

Plot: rho effect on convergence

```
figure('Name','Effect of rho','Position',[50 50 1000 400]);

subplot(1,2,1);
iters_vals = [rho_results.iters];
semilogx([rho_results.rho], iters_vals, 'rs-', 'LineWidth', 2, 'MarkerSize',
8);
xlabel('\rho'); ylabel('Iterations to convergence');
title('Convergence speed vs \rho');
grid on;

subplot(1,2,2);
colors = lines(length(rhos));
hold on;
for i = 1:length(rhos)
    semilogy(rho_results(i).r_norm, 'Color', colors(i,:), 'LineWidth', 1.5);
end
xlabel('Iteration'); ylabel('Primal residual ||r||_2');
title('Convergence for different \rho values');
legend(arrayfun(@(r) sprintf('\\rho=%.2g',r), rhos, 'UniformOutput',false),
...
    'Location','northeast');
grid on;

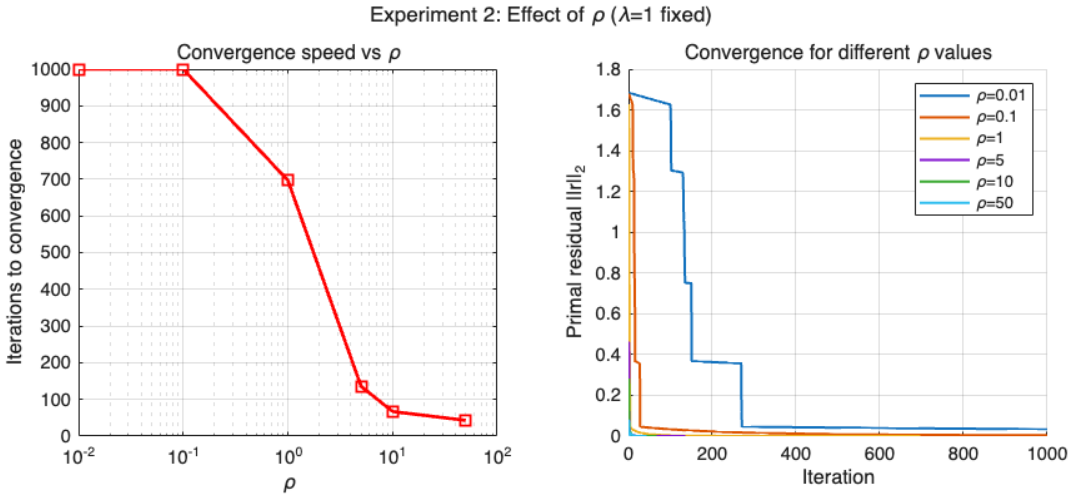
sgtitle('Experiment 2: Effect of \rho (\lambda=1 fixed)');
saveas(gcf, 'experiment_rho.png');
fprintf('Figure saved: experiment_rho.png\n');

fprintf('\n================================================\n');
fprintf('SUMMARY:\n');
```

```
fprintf('  - Larger lambda => sparser solution x (fewer nonzeros)\n');
fprintf('  - rho does NOT change the optimal value\n');
fprintf('  - rho too small or too large => slow convergence\n');
fprintf('  - rho near 1 (or adaptively chosen) often works best\n');
fprintf('================================================\n');
```

```
Figure saved: experiment_rho.png
```

```
================================================
SUMMARY:
  - Larger lambda => sparser solution x (fewer nonzeros)
  - rho does NOT change the optimal value
  - rho too small or too large => slow convergence
  - rho near 1 (or adaptively chosen) often works best
================================================
```



Published with MATLAB® R2025b

ρ	Objective value	$\text{nnz}(\lvert x_i \rvert > 10^{-4})$	Iterations
0.01	3.5476×10^0	19	1000
0.1	3.5730×10^0	20	1000
1	3.5773×10^0	6	697
5	3.5773×10^0	6	136
10	3.5773×10^0	6	67
50	3.5773×10^0	6	43

Table 2: Effect of ρ on convergence (with $\lambda = 1$).